

Proyecto de grado

Capítulo IV

Diseño y desarrollo de la solución

Dr. Carlos A. Jacomino

2026

CAPÍTULO IV. DISEÑO Y DESARROLLO DE LA SOLUCIÓN

1 Arquitectura general del sistema

2 Requerimientos funcionales y no funcionales

3 Modelado UML

- 3.1 Diagrama de casos de uso
- 3.2 Diagrama de actividades
- 3.3 Diagrama de clases
- 3.4 Diagrama de secuencia
- 3.5 Diagrama de componentes
- 3.6 Diagrama de despliegue
- 3.7 Diagrama de estados, cuando corresponda

4 Diseño de interfaz

5 Implementación

6 Pruebas del sistema



¿Qué es un requerimiento funcional y no funcional?



Requerimiento funcional

Describe **lo que el sistema debe hacer**.



¿Qué debe hacer el sistema?

Ejemplos:



Permitir iniciar sesión



Registrar usuarios



Mostrar reportes



Requerimiento no funcional

Describe **cómo debe funcionar** el sistema o **qué calidad debe cumplir**.



¿Cómo debe funcionar el sistema?

Ejemplos:



Cargar en menos de 3 segundos



Proteger la información del usuario



Ser fácil de usar



Funcional = lo que el sistema hace



No funcional = cómo debe hacerlo

Tips para redactar requerimientos de manera excelente

Requerimientos funcionales y no funcionales

1

USAR UNA ESTRUCTURA UNIFORME

Se recomienda iniciar con **"El sistema deberá..."** para mantener claridad y consistencia. Evitar frases vagas como "podría" o "se espera que".

 **Frase vaga (incorrecta):**
"El sistema podría permitir registrar usuarios."



 **Ejemplo correcto:**
"El sistema deberá permitir registrar usuarios."

2

REDACTAR UNA SOLA EXIGENCIA POR REQUERIMIENTO

Cada requerimiento debe describir una única función o condición.

 **Incorrecto:**
"El sistema deberá registrar, autenticar y recuperar la contraseña de usuarios."

 **Correcto (separar en varios):**

- RF-01. El sistema deberá registrar usuarios.
- RF-02. El sistema deberá autenticar usuarios.
- RF-03. El sistema deberá permitir recuperar la contraseña de usuarios.



3

SER CONCRETO Y MEDIBLE

Evitar términos vagos como rápido, fácil, eficiente, amigable o adecuado si no son medibles.

 **Incorrecto:**
"El sistema deberá ser rápido."

 **Correcto (medible):**
"El sistema deberá mostrar la pantalla principal en un tiempo máximo de 3 segundos."



4

DIFERENCIAR FUNCIÓN DE CALIDAD

Un requerimiento puede ser funcional (**qué hace el sistema**) o **no funcional** (**cómo debe hacerlo o qué calidad debe cumplir**).

Funcional (qué hace)

Define la función o acción del sistema.



Ejemplo:

"El sistema deberá permitir al usuario visualizar un recorrido 360°."

No funcional (cómo debe hacerlo)

Define la calidad, rendimiento o condición que debe cumplir.



Ejemplo:

"Las escenas 360° deberán cargarse en un máximo de 3 segundos."

5

DEFINIR EL ACTOR CUANDO SEA NECESARIO

Indicar quién realiza la acción (usuario, administrador, sistema u otro) cuando sea relevante para el contexto del requerimiento.



Usuario



Administrador



Sistema

Ejemplo:

"El administrador deberá poder crear, modificar y desactivar usuarios."

6

USAR VERBOS OBSERVABLES

Emplear verbos claros que indiquen acciones verificables. Estos verbos ayudan a que el requerimiento sea comprobable.

Verbos recomendados:

- registrar
- consultar
- generar
- validar
- almacenar
- actualizar
- eliminar
- mostrar
- autenticar
- exportar



7

VERIFICAR QUE SEA COMPROBABLE

Todo requerimiento debe ser demostrable mediante pruebas y debe tener un criterio de aceptación.

Ejemplo de requerimiento:

RF-08. El sistema deberá permitir generar un reporte de usuarios registrados, filtrado por fecha, y exportarlo en formato PDF.



Criterio de aceptación:

"Se considerará cumplido cuando el reporte pueda generarse y exportarse correctamente."

Ejemplo de Requerimientos de un Sistema de Información

(Formato académico para tesis de Ingeniería en Sistemas)

Contexto del proyecto (ejemplo)

Nombre del sistema

Sistema de Gestión de Reservas de Espacios Académicos

Problema que resuelve

La gestión manual de reservas genera conflictos de horarios, doble asignación y falta de control en el uso de los espacios.

Usuarios/actores

- Estudiante
- Docente
- Administrador
- Coordinador académico

Módulos principales

- Autenticación
- Gestión de espacios
- Reservas
- Reportes

Tecnologías / restricciones

- Aplicación web
- Base de datos PostgreSQL
- Interfaz responsiva
- Despliegue en servidor institucional

REQUERIMIENTOS FUNCIONALES (RF)

Funciones que el sistema debe realizar

Código	Descripción del requerimiento	Prioridad	Fuente
Módulo 1. Autenticación y usuarios			
RF-01	El sistema debe permitir al usuario iniciar sesión con sus credenciales institucionales.	Alta	Entrevista (usuarios)
RF-02	El sistema debe permitir al administrador registrar, modificar y desactivar usuarios.	Alta	Entrevista (administrador)
Módulo 2. Gestión de espacios			
RF-03	El sistema debe permitir al administrador registrar y actualizar la información de los espacios académicos.	Alta	Entrevista (administrador)
RF-04	El sistema debe permitir consultar la disponibilidad de un espacio en un rango de fechas.	Alta	Entrevista (docente/estudiante)
Módulo 3. Reservas			
RF-05	El sistema debe permitir al usuario realizar una reserva de espacio cuando esté disponible.	Alta	Entrevista (usuarios)
RF-06	El sistema debe permitir al usuario cancelar una reserva antes de la fecha de uso.	Media	Entrevista (usuarios)
RF-07	El sistema debe enviar una notificación de confirmación de la reserva al usuario.	Media	Entrevista (usuarios)
Módulo 4. Reportes			
RF-08	El sistema debe generar reportes de uso de espacios por período y por facultad.	Media	Entrevista (coordinador)
RF-09	El sistema debe permitir exportar los reportes en formato PDF y Excel.	Baja	Entrevista (coordinador)

REQUERIMIENTOS NO FUNCIONALES (RNF)

Condiciones de calidad, rendimiento y operación del sistema

Código	Categoría	Descripción del requerimiento	Prioridad	Fuente
RNF-01	 Rendimiento	El sistema debe responder en menos de 2 segundos ante el 95% de las solicitudes en condiciones normales de uso.	Alta	Norma (ISO 25010)
RNF-02	 Seguridad	El sistema debe cifrar las contraseñas de los usuarios y almacenarlas mediante un algoritmo de hash seguro.	Alta	Norma (OWASP)
RNF-03	 Usabilidad	La interfaz debe ser intuitiva y permitir que un usuario nuevo complete una reserva en menos de 3 minutos.	Alta	Entrevista (usuarios)
RNF-04	 Disponibilidad	El sistema debe estar disponible el 99% del tiempo durante el período académico.	Alta	Norma (ISO 25010)
RNF-05	 Escalabilidad	El sistema debe soportar un incremento del 50% de usuarios concurrentes sin degradación significativa del rendimiento.	Media	Proyección del proyecto
RNF-06	 Mantenibilidad	El código del sistema debe estar documentado y seguir un estándar de codificación para facilitar su mantenimiento.	Media	Norma (IEEE 830)
RNF-07	 Portabilidad	El sistema debe ser compatible con navegadores modernos (Chrome, Edge y Firefox) en sus versiones vigentes.	Media	Entrevista (usuarios)
RNF-08	 Compatibilidad	El sistema debe adaptarse a dispositivos de escritorio, tableta y teléfono móvil (diseño responsivo).	Media	Entrevista (usuarios)



Trazabilidad con Casos de Uso e Historias de Usuario

- Los requerimientos funcionales (RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07, RF-08 y RF-09) pueden derivarse directamente en Casos de Uso o Historias de Usuario, ya que describen funcionalidades concretas del sistema.
- Los requerimientos no funcionales (RNF) complementan los Casos de Uso, estableciendo condiciones de calidad que deben validarse en las pruebas del sistema.

Elementos para presentar un UML en la tesis

Elemento	Qué debe contener
Nombre del UML	Tipo de diagrama
Objetivo	Qué aspecto del sistema modela
Figura	Diagrama legible y completo
Descripción	Explicación de actores, objetos, procesos o componentes
Interpretación	Qué demuestra respecto al funcionamiento del sistema
Correspondencia	Relación con el software realmente implementado

UML y sus principales diagramas

Ingeniería en Sistemas

Qué significa UML, para qué sirve y cuáles son los diagramas más usados

Material de estudio

Diagrama de casos de uso

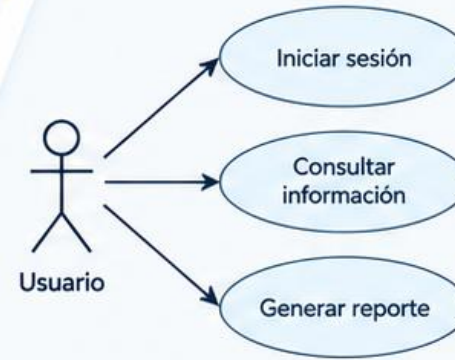


Diagrama de clases

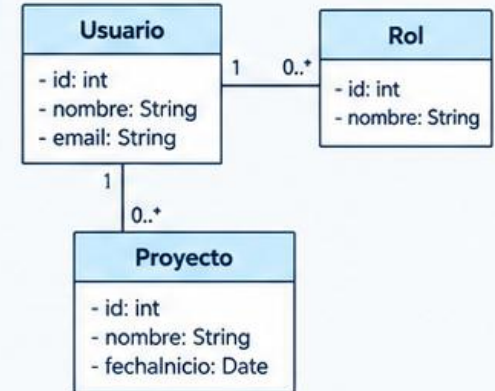


Diagrama de secuencia

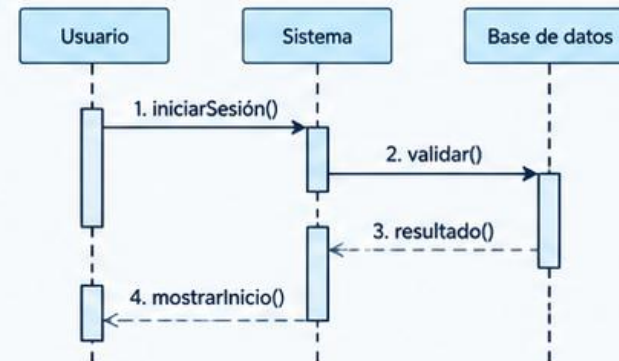


Diagrama de actividades

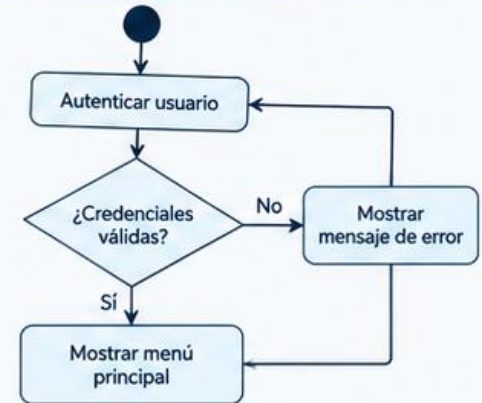


Diagrama de despliegue



¿Qué significa UML?

Unified Modeling Language

Material de estudio



= Unified = Unificado

Significa que es un lenguaje estándar y comúnmente aceptado.



= Modeling = Modelado

Se refiere a la representación de sistemas mediante modelos.



= Language = Lenguaje

Indica que es un conjunto de reglas y notaciones para comunicar modelos.

UML es el Lenguaje Unificado de Modelado. Sirve para **analizar, diseñar y documentar** sistemas de software.



No es un lenguaje de programación;
es un lenguaje visual de modelado.

Diagrama de casos de uso

Representa a los actores y las funciones principales del sistema.



Qué muestra:

actores, casos de uso y alcance del sistema.



Para qué sirve:

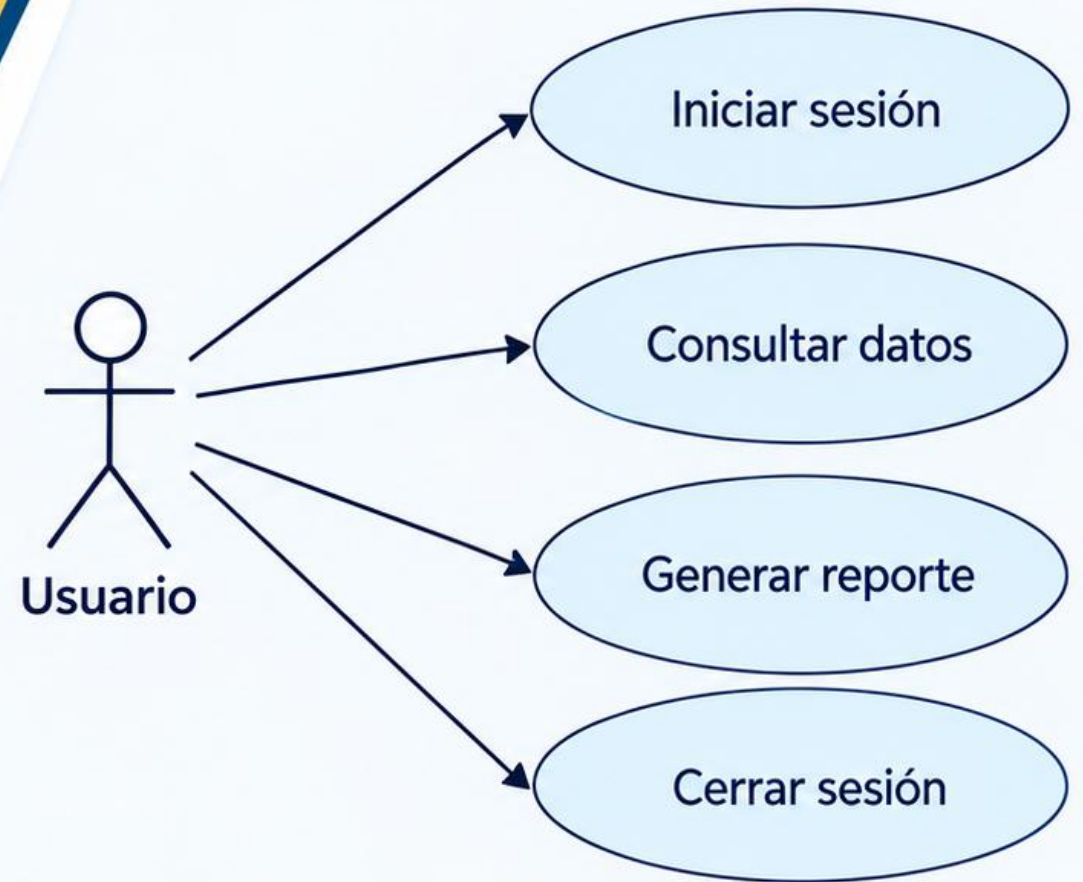
identificar requisitos funcionales.



Cuándo se usa:

al inicio del análisis del sistema.

Ejemplo de diagrama de casos de uso



Ejemplo: un usuario interactúa con las funciones del sistema.

Diagrama de clases

Describe la estructura del sistema mediante clases, atributos, métodos y relaciones.



Qué muestra:
clases y relaciones.



Para qué sirve:
diseñar la base lógica del software.



Elementos clave:
atributos, métodos, asociación, herencia y multiplicidad.

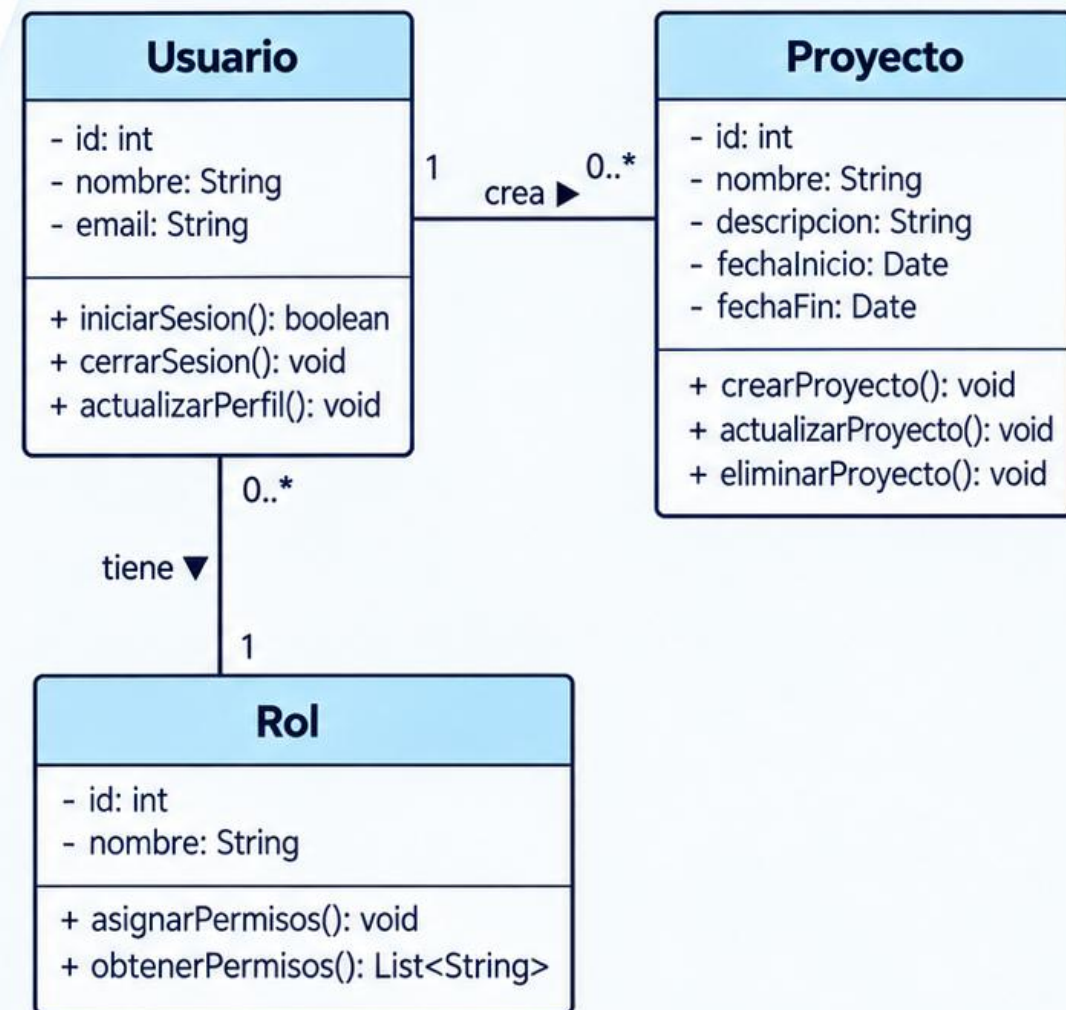


Diagrama de secuencia

Muestra el orden temporal de los mensajes intercambiados entre los elementos del sistema.



Qué muestra:

interacción paso a paso.



Para qué sirve:

detallar escenarios de funcionamiento.



Elementos clave:

actores, objetos, mensajes y tiempo.

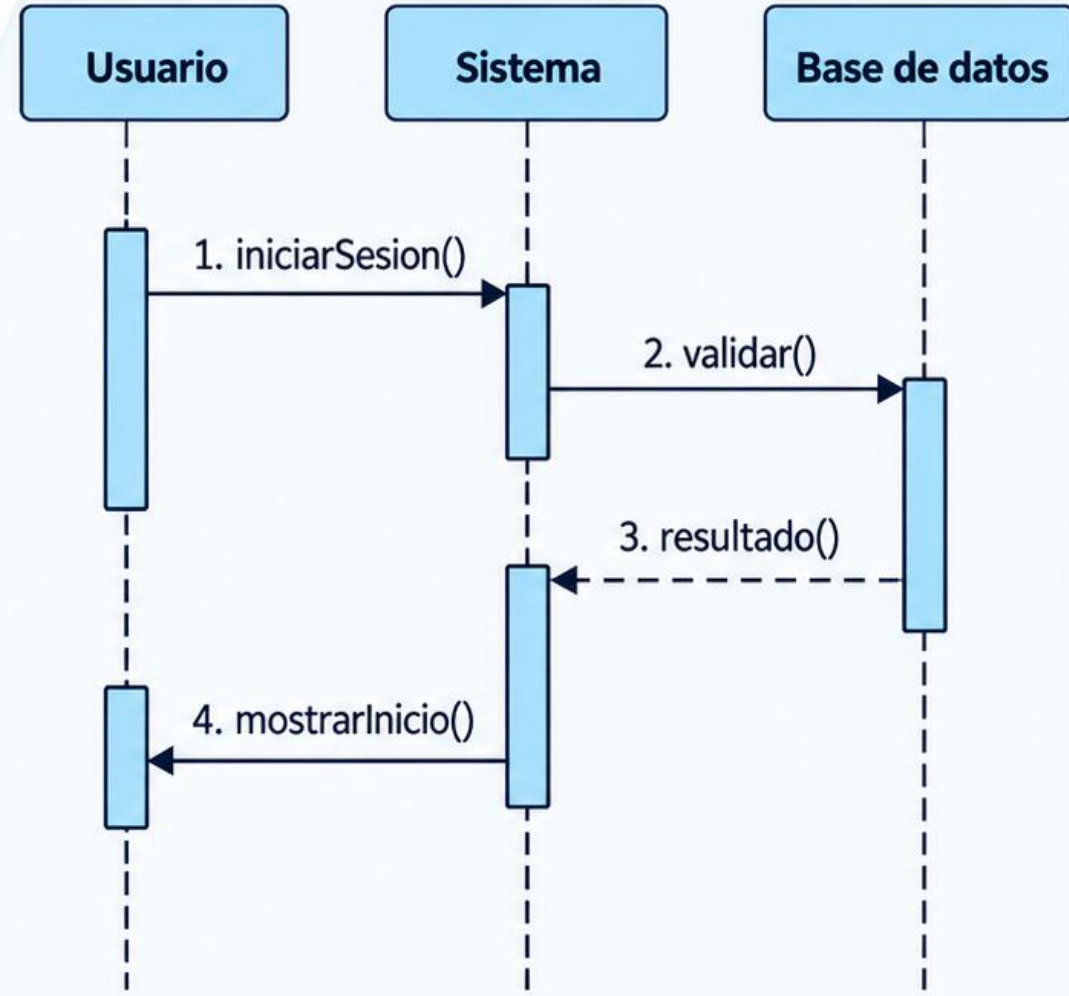


Diagrama de actividades

Representa el flujo de trabajo, las decisiones y la secuencia de acciones de un proceso.



Qué muestra:

actividades, decisiones e inicios/finales.



Para qué sirve:

modelar procesos, algoritmos y flujos funcionales.



Elementos clave:

acción, decisión, bifurcación y flujo.

Ejemplo de diagrama de actividades

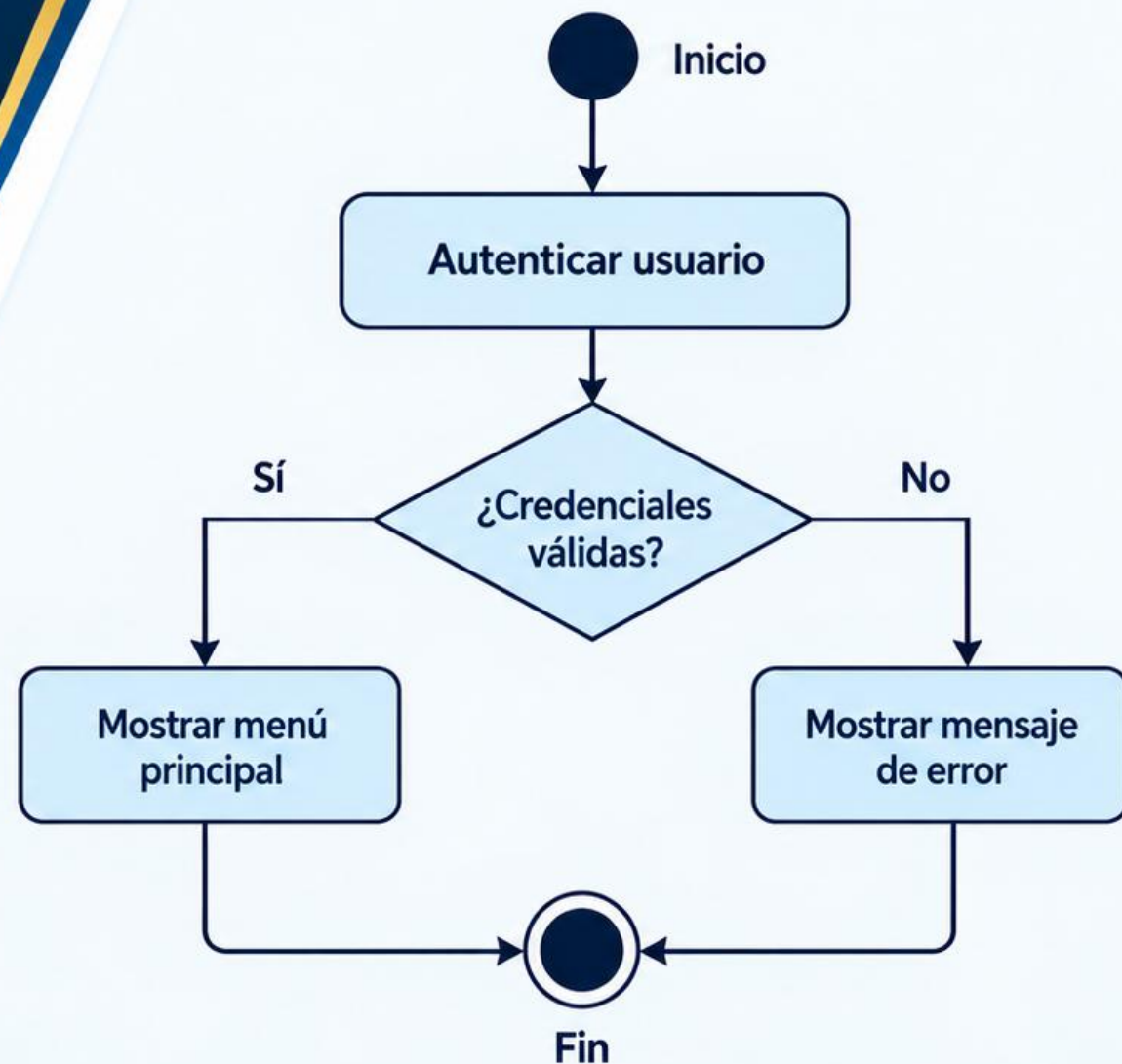


Diagrama de componentes

Muestra los módulos principales del sistema y cómo se relacionan entre sí.



Qué muestra:

componentes, interfaces y dependencias.



Para qué sirve:

representar la arquitectura del software.



Cuándo se usa:

en el diseño técnico y la integración de módulos.

Diagrama de componentes (ejemplo)

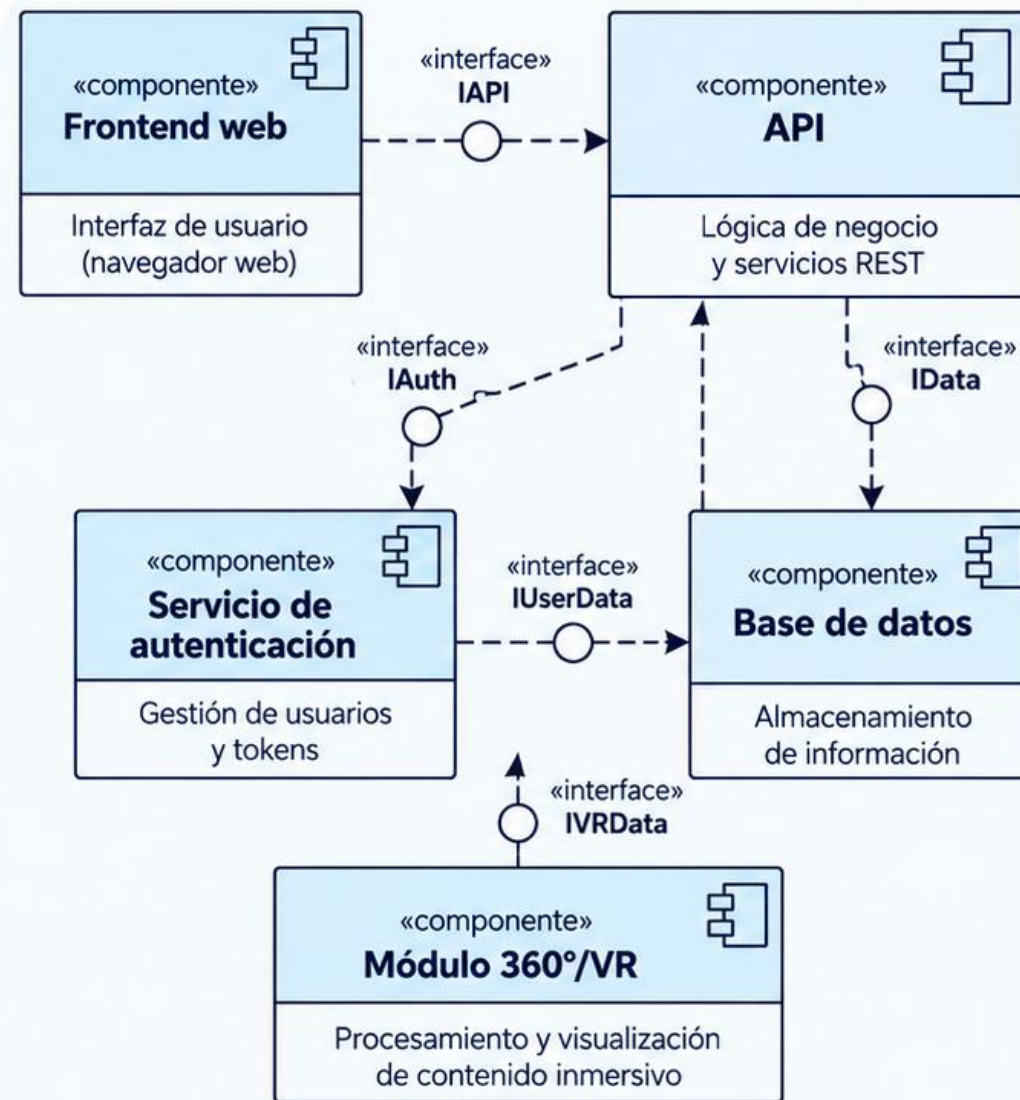
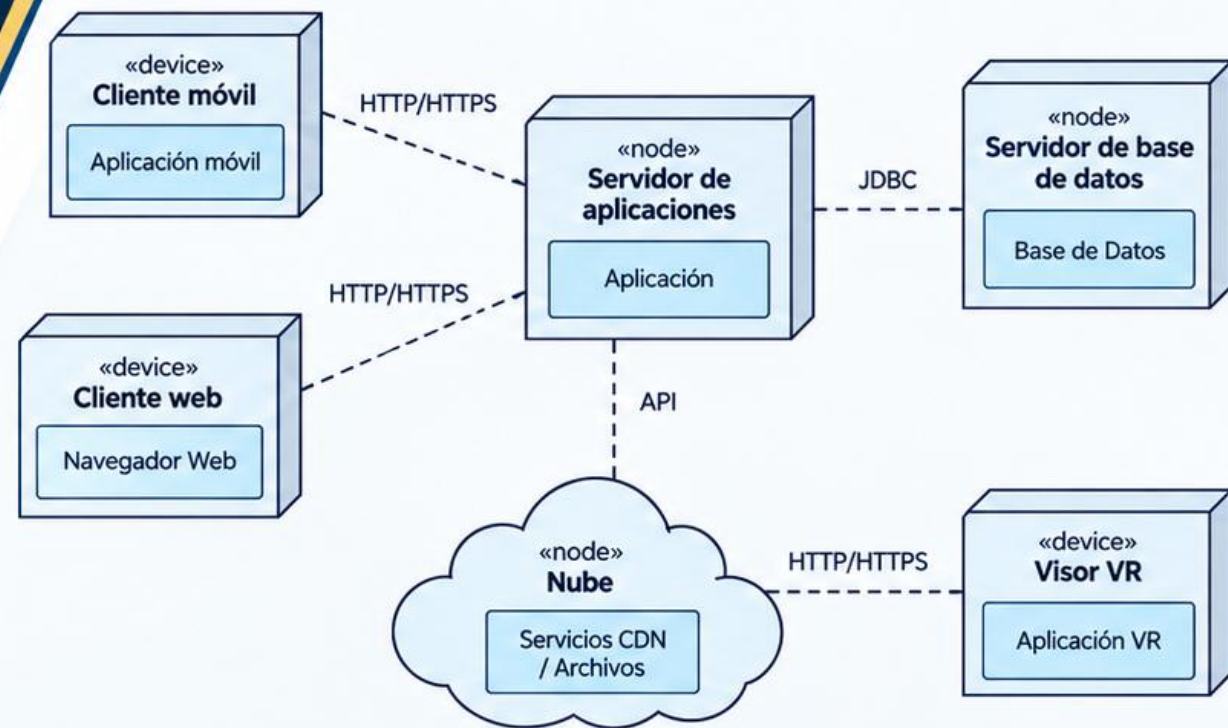


Diagrama de despliegue

Representa dónde se ejecuta el sistema: dispositivos, servidores y conexiones.

Ejemplo de diagrama de despliegue



Qué muestra:

nodos físicos o virtuales.



Para qué sirve:

documentar la infraestructura tecnológica.



Elementos clave:

cliente, servidor, red, base de datos y dispositivos.

Diagrama de estados

Describe los cambios de estado de un objeto, pantalla o proceso y lo largo del tiempo.



Qué muestra:
estados y transiciones.

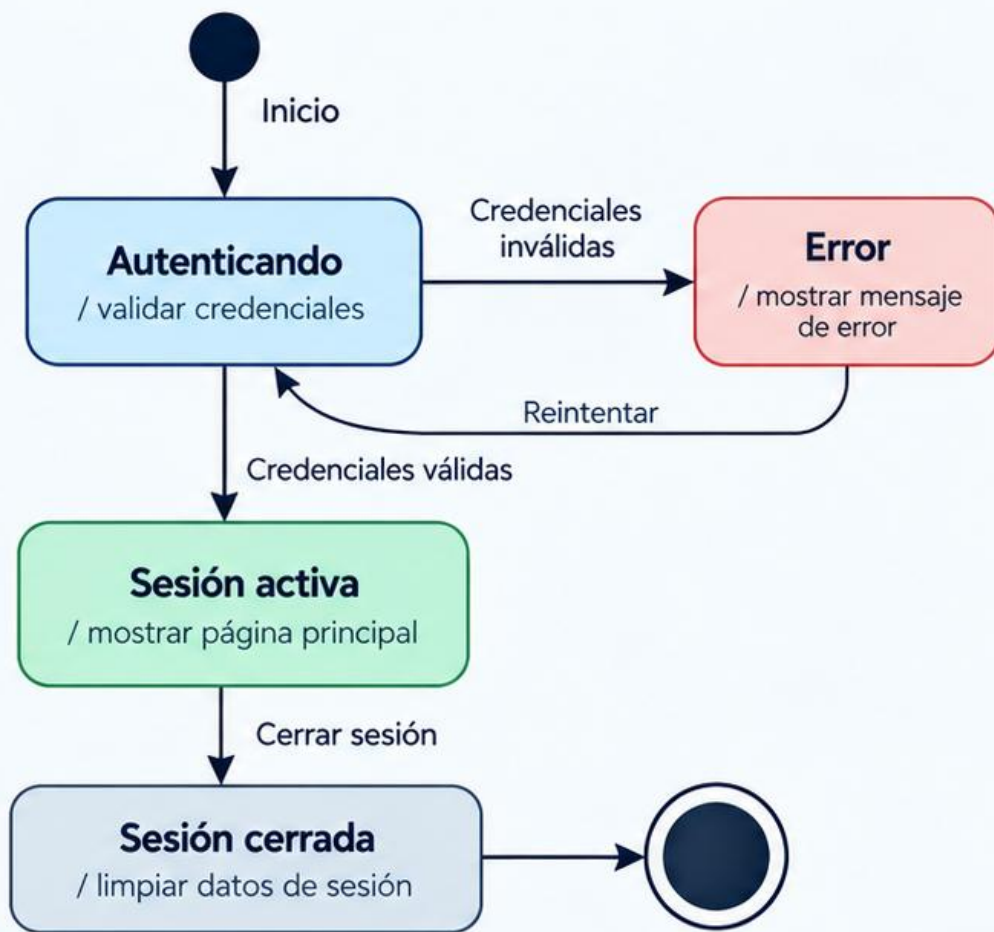


Para qué sirve:
modelar comportamientos dinámicos.



Cuándo se usa:
cuando un elemento cambia según eventos o condiciones.

Ejemplo: Sistema de inicio de sesión



Muy útil en apps, videojuegos, sistemas web y experiencias inmersivas.